

MegaPOS

Sistema de Punto de Venta — Módulo Java / Eclipse

Instituto Tecnológico

Materia: Programación Orientada a Objetos

Integrante 1: Ángel Adrián

Integrante 2: Felipe González

Integrante 3: Kevin David

Mayo 2026

Índice

Introducción.....	3
PARTE I — Arquitectura del Proyecto (Ángel Adrián)	4
1.1 Tecnologías y herramientas utilizadas	4
1.2 Estructura de paquetes.....	4
1.3 Patrón de diseño MVC.....	5
1.4 Modelos de datos (clases del paquete model).....	5
PARTE II — Lógica de Negocio y Controladores (Felipe González)	7
2.1 LoginController — Acceso al sistema	7
2.2 VentaController — Proceso de venta.....	8
2.3 InventarioController — Gestión de productos	9
2.4 CompraController — Entradas de proveedor.....	10
2.5 DevolucionController — Devoluciones	10
2.6 DescuentoController — Descuentos.....	11
PARTE III — Utilidades, Reportes y Cierre (Kevin David)	12
3.1 ConexionBD — Puente con MySQL	12
3.2 LectorJSON — Carga de catálogo desde archivo.....	12
3.3 ManejadorErrores — Gestión centralizada de errores.....	13
3.4 ReporteExcel — Exportación de ventas a CSV	13
3.5 AuditoriaController — Visor de tickets.....	13
3.6 CierreCajaController — Cierre del turno.....	14
Conclusión	15

Introducción

El presente documento describe el desarrollo del módulo de aplicación del sistema de punto de venta MegaPOS, construido con Java en el entorno de desarrollo Eclipse. El proyecto forma parte de la materia de Programación Orientada a Objetos y complementa la base de datos MySQL que se diseñó en una etapa previa.

MegaPOS es un sistema orientado a una tienda departamental que permite gestionar ventas, inventario, compras a proveedores, devoluciones, descuentos y cierres de caja, todo desde una interfaz gráfica desarrollada con la biblioteca Swing de Java.

La aplicación sigue el patrón de diseño Modelo-Vista-Controlador (MVC), separando claramente la lógica de negocio, la presentación visual y la gestión de datos. Para la comunicación con la base de datos se utiliza JDBC a través del conector oficial de MySQL.

El equipo de trabajo está integrado por Ángel Adrián, Felipe González y Kevin David. Cada integrante asumió de manera equitativa una parte del sistema, tal como se describe a lo largo de este documento.

PARTE I

Arquitectura del Proyecto

Elaborado por: Ángel Adrián

1.1 Tecnologías y herramientas utilizadas

Para el desarrollo de MegaPOS utilizamos las siguientes tecnologías:

Tecnología / Herramienta	Versión / Descripción	Uso en el proyecto
Java SE	JDK 17+	Lenguaje principal del sistema
Eclipse IDE	2024 o superior	Entorno de desarrollo integrado
Swing	Incluida en Java SE	Interfaz gráfica de usuario
JDBC	API estándar de Java	Comunicación con MySQL
mysql-connector-j	8.x / 9.x	Driver de conexión a la BD
MySQL Server	9.5	Motor de base de datos
JSON (manual)	Propio / LectorJSON.java	Carga inicial de catálogo de productos

1.2 Estructura de paquetes

El proyecto está organizado en cuatro paquetes principales, siguiendo las convenciones del patrón MVC:

Paquete	Clases que contiene	Responsabilidad
main.model	Producto, Venta, DetalleVenta, Usuario	Clases de datos (entidades del negocio)
main.controller	VentaController, InventarioController, LoginController, CompraController, DevolucionController, DescuentoController, AuditoriaController, CierreCajaController	Lógica de negocio y eventos de la UI
main.view	MainView, LoginView	Interfaz gráfica (ventanas Swing)
main.util	ConexionBD, LectorJSON, ManejadorErrores, ReporteExcel	Utilidades y servicios transversales
main.dao	VentaDAO, ProductoDAO, CompraDAO, DevolucionDAO, CajaDAO, DescuentoDAO	Acceso a datos — consultas SQL a MySQL

1.3 Patrón de diseño MVC

MVC (Modelo-Vista-Controlador) es el patrón de diseño que adoptamos para organizar el código de MegaPOS. La idea central es que cada capa tiene una responsabilidad única y no debe mezclarse con las demás:

El Modelo representa los datos del negocio: qué es un Producto, qué campos tiene una Venta, cómo se compone un DetalleVenta. Estas clases no hacen consultas SQL ni tocan la pantalla; solo almacenan información.

La Vista es todo lo que ve el usuario. En nuestro caso, LoginView es la ventana de inicio de sesión y MainView es la ventana principal que contiene todos los paneles del sistema (ventas, inventario, compras, devoluciones, etc.). La vista no toma decisiones; solo muestra y recibe datos.

El Controlador es quien coordina todo. Por ejemplo, VentaController escucha cuando el cajero presiona el botón 'Cobrar', valida los datos, arma el objeto Venta, llama al DAO para guardarlo en MySQL y actualiza la pantalla con el resultado. Cada funcionalidad del sistema tiene su propio controlador.

1.4 Modelos de datos (clases del paquete model)

Las clases del paquete main.model representan las entidades principales del negocio. Cada una encapsula sus atributos con métodos get/set y se usa para transportar datos entre las capas del sistema.

Clase	Atributos principales	Propósito
Producto	codigo, nombre, precioCompra, precioVenta, stock, limiteStockMinimo, categoria, imagenRuta	Representa un artículo del catálogo de la tienda
Venta	folio, subtotal, descuento, total, metodoPago, montoRecibido, cambio, fecha, hora, List<DetalleVenta>	Representa una transacción de venta completa con todos sus detalles
DetalleVenta	producto (Producto), cantidad	Representa un renglón de la venta: qué producto y cuántas unidades
Usuario	id, nombre, username, password, rol	Representa a un operador del sistema (ADMIN o CAJERO)

La clase Venta merece una mención especial porque contiene una lista de objetos DetalleVenta, lo que refleja directamente la relación entre las tablas ventas y detalle_ventas de la base de datos. Cuando guardamos una venta, primero insertamos el registro en ventas y después iteramos la lista de detalles para insertar cada renglón en detalle_ventas.

PARTE II

Lógica de Negocio y Controladores

Elaborado por: Felipe González

2.1 LoginController — Acceso al sistema

LoginController es el primer controlador que se ejecuta cuando arranca la aplicación. Su responsabilidad es autenticar al operador y, según el rol que se seleccione, inicializar los controladores correspondientes.

El sistema tiene dos roles: CAJERO y ADMIN. El rol CAJERO no requiere contraseña y abre el sistema con un fondo de caja de \$1,000. El rol ADMIN requiere la contraseña '1234' y tiene acceso a funciones adicionales como compras a proveedor, descuentos y un fondo de \$100,000.

Acción del usuario	Qué hace el controlador
Clic en 'Cajero'	Llama a abrirSistema('CAJERO', 1000.0) sin pedir contraseña
Clic en 'Admin'	Muestra el campo de contraseña y espera la confirmación
Contraseña correcta	Abre MainView, instancia todos los controladores necesarios y los registra
Contraseña incorrecta	Muestra mensaje de error y limpia el campo para reintentar

Una vez autenticado, LoginController instancia VentaController, InventarioController, DevolucionController, AuditoriaController y CierreCajaController para cualquier rol. CompraController y DescuentoController solo se instancian si el rol es ADMIN, porque esas funcionalidades son exclusivas del administrador.

2.2 VentaController — Proceso de venta

VentaController es el controlador más complejo del sistema. Gestiona todo el flujo de una venta: mostrar el catálogo visual de productos, agregar artículos al carrito, aplicar descuentos, procesar el cobro y generar el ticket.

El catálogo se muestra como un grid de tarjetas visuales (paneles redondeados de 155x195 px). Cada tarjeta muestra la imagen del producto, su nombre, el stock disponible y el precio de venta. Si el stock está por debajo del mínimo configurado, la etiqueta de stock aparece en rojo con la leyenda '¡STOCK BAJO!'. El cajero puede filtrar por categoría o buscar por nombre/código.

Paso	Método involucrado	Descripción
1	cargarInventarioDesdeBD()	Obtiene todos los productos desde MySQL vía ProductoDAO
2	renderizarGridVisual()	Dibuja las tarjetas de producto en la pantalla
3	agregarProductoDirecto()	Agrega el producto al carrito validando stock disponible
4	actualizarTotal()	Recalcula subtotal, aplica descuento y muestra el total
5	evaluarMetodoPago()	Ajusta campos según si el pago es EFECTIVO o TARJETA
6	procesarCobroYGenerarTicket()	Valida montos, guarda en BD, descuenta stock y emite ticket

El folio de cada venta se genera automáticamente con 'TK-' más el timestamp en milisegundos (`System.currentTimeMillis()`), lo que garantiza que no haya dos folios iguales. Después de cada venta exitosa, el carrito se vacía y el inventario se refresca desde la base de datos.

`VentaController` también gestiona los reportes: por fecha específica, por intervalo de fechas, reporte de productos más vendidos y reporte de stock bajo. Todos los reportes se pueden exportar a CSV con un `JFileChooser`.

2.3 InventarioController — Gestión de productos

`InventarioController` permite al operador (principalmente al ADMIN) ver, agregar, modificar y eliminar productos del catálogo. La lista de productos se muestra en una `JTable` con filtros de búsqueda por texto y por categoría.

Cuando el usuario hace clic en una fila de la tabla, los datos del producto se cargan automáticamente en los campos del formulario. La columna 'Estado' muestra 'STOCK INSUFICIENTE' en rojo si el stock del producto es igual o menor al límite mínimo configurado.

Operación	Validación aplicada
Agregar producto	Verifica que no exista otro con el mismo ID, que precio y stock no sean negativos
Modificar producto	Requiere que el producto exista y que precio/stock sean válidos
Eliminar producto	Valida que el producto exista y evita eliminar registros que puedan afectar el control del inventario.
Seleccionar imagen	Usa <code>JFileChooser</code> y guarda o utiliza una ruta relativa dentro de la carpeta Productos, organizada por categoría y nombre del archivo.

2.4 CompraController — Entradas de proveedor

`CompraController` es exclusivo del rol ADMIN. Permite registrar las entradas de mercancía provenientes de proveedores: se indica el código del producto, el nombre del proveedor, la cantidad recibida y el precio de compra unitario. Al guardar, el stock del producto se incrementa automáticamente en la base de datos.

También permite cancelar una entrada previamente registrada: al ingresar el ID de la compra y confirmar, el sistema resta del stock la cantidad correspondiente y marca la entrada como cancelada. El historial de todas las entradas se muestra en un área de texto dentro del mismo panel.

2.5 DevolucionController — Devoluciones

`DevolucionController` gestiona las devoluciones de productos ya vendidos. Para registrar una devolución, el cajero ingresa el folio de la venta original, el código del producto a devolver, la cantidad y el motivo.

El controlador aplica varias validaciones antes de registrar: verifica que el producto pertenezca a la venta indicada, que no se haya devuelto ya completamente, y que la cantidad a devolver no supere las unidades disponibles (vendidas menos ya devueltas). Si todo es correcto, el stock del producto se restaura y el reembolso queda registrado.

2.6 DescuentoController — Descuentos

DescuentoController permite al ADMIN administrar los descuentos configurables del sistema. Cada descuento tiene: nombre, ámbito (PRODUCTO o PROVEEDOR), tipo (PORCENTAJE, MONTO_FIJO, DOS_POR_UNO o TRES_POR_DOS), valor y estado activo/inactivo. Los descuentos se almacenan en la tabla descuentos de MySQL.

Al inicializarse, el controlador llama a asegurarTablaYDatosBase() para garantizar que la tabla exista y tenga al menos los descuentos base del sistema. El ADMIN puede crear nuevos descuentos, editar los existentes, activarlos/desactivarlos o eliminarlos desde una JTable interactiva.

PARTE III

Utilidades, Reportes y Cierre de Caja

Elaborado por: Kevin David

3.1 ConexionBD — Puente con MySQL

La clase ConexionBD (paquete main.util) centraliza toda la configuración de conexión a la base de datos. Tiene un único método estático: `getConexion()`, que devuelve un objeto `Connection` listo para usar.

Parámetro	Valor configurado	Descripción
URL	<code>jdbc:mysql://localhost:3306/mega_pos_db?useSSL=false&serverTimezone=UTC</code>	Dirección y nombre de la base de datos
Usuario	<code>root</code>	Usuario del servidor MySQL
Contraseña	<code>123</code>	Contraseña del servidor local
Driver	<code>com.mysql.cj.jdbc.Driver</code>	Clase del conector oficial de MySQL para Java

El método primero carga el driver con `Class.forName()`, que registra el conector en la JVM. Después llama a `DriverManager.getConnection()` con la URL, el usuario y la contraseña. Si ocurre algún error (driver no encontrado o fallo de conexión), se delega al `ManejadorErrores` para mostrar el mensaje al usuario y se devuelve `null`.

Para que `ConexionBD` funcione, es obligatorio agregar el archivo `mysql-connector-j-x.x.jar` al Build Path del proyecto en Eclipse. Sin ese archivo, la aplicación no puede comunicarse con MySQL y lanza una `ClassNotFoundException`.

3.2 LectorJSON — Carga de catálogo desde archivo

`LectorJSON` es una clase utilitaria que permite cargar y guardar la lista de productos desde un archivo `productos.json` almacenado localmente. Fue diseñada como mecanismo alternativo de persistencia, independiente de MySQL.

El método `leerProductos()` lee el archivo, lo divide en objetos separados por `},{'` y extrae cada campo con búsqueda de posiciones de cadena (sin librerías externas). El método `guardarProductos()` reescribe el archivo completo con el estado actual del inventario, escapando las rutas de imagen para que sean válidas en JSON (barras dobles).

Si el archivo no existe o hay algún error de lectura, la clase llama a `ManejadorErrores.registrarError()` y devuelve una lista vacía, sin interrumpir la ejecución del sistema.

3.3 ManejadorErrores — Gestión centralizada de errores

`ManejadorErrores` es una clase muy simple pero fundamental: tiene un único método estático `registrarError(mensajeUsuario, exception)` que hace dos cosas: imprime el error detallado en la consola (`System.err + printStackTrace`) y muestra al usuario un diálogo de alerta con `JOptionPane`, con el mensaje legible y el detalle técnico.

Al centralizar el manejo de errores en una sola clase, todos los controladores y utilidades del proyecto muestran los mensajes de error de la misma forma, sin duplicar código. Si en el futuro quisiéramos agregar un log a archivo, solo habría que modificar esta clase.

3.4 ReporteExcel — Exportación de ventas a CSV

ReporteExcel.java genera archivos CSV (Comma-Separated Values) con el historial de ventas. Aunque el nombre de la clase sugiere Excel, optamos por CSV en lugar de XLSX para evitar depender de librerías externas como Apache POI, que habrían aumentado el tamaño y la complejidad del proyecto.

El sistema genera archivos CSV con información de ventas consultada desde MySQL. Estos reportes pueden incluir folio, fecha, hora, subtotal, descuento, total, método de pago, monto recibido y cambio. Los archivos CSV generados pueden abrirse directamente en Microsoft Excel o Google Sheets.

3.5 AuditoriaController — Visor de tickets

AuditoriaController gestiona el panel de auditoría, que permite consultar y reimprimir cualquier ticket generado previamente. Al abrir el panel, el controlador carga todos los folios disponibles en un JComboBox. Cuando el usuario selecciona un folio, se consulta en MySQL el texto completo del ticket guardado en la venta; si no existe ese texto, el sistema intenta reconstruir el ticket usando las tablas ventas, detalle_ventas y productos.

Para evitar que el ComboBox dispare eventos mientras se están cargando los folios, usamos una bandera booleana cargandoFolios que bloquea temporalmente el listener. El área de texto se configura con fondo oscuro, texto blanco y fuente Monospaced a 14pt para simular la apariencia de un ticket físico en pantalla.

3.6 CierreCajaController — Cierre del turno

CierreCajaController gestiona el proceso de cierre al final del turno del cajero. Cuando el operador presiona el botón 'Cerrar Caja', el controlador muestra un diálogo de confirmación que resume lo que incluirá el reporte: ventas del día, tickets generados, caja inicial y total final.

Si el operador confirma, se llama a CajaDAO.generarCierreCaja() que consulta MySQL y arma el reporte en texto. El reporte se muestra en una ventana con área de texto scrolleable (520x420 px). Después de mostrar el reporte, la aplicación cierra la ventana principal (MainView) y vuelve automáticamente a la pantalla de LoginView, lista para el siguiente turno.

Dato en el reporte de cierre	Origen
Caja inicial	Parámetro recibido al instanciar el controlador (según rol)
Total de ventas	Suma de todos los totales del día consultada en MySQL
Número de tickets	Conteo de folios del día en la tabla ventas
Total final en caja	Caja inicial + total de ventas en efectivo
Rol que cerró caja	Parámetro 'rol' recibido al instanciar (ADMIN o CAJERO)

Conclusión

El desarrollo de MegaPOS nos permitió aplicar de forma práctica los conceptos de la programación orientada a objetos en un sistema real y funcional. A lo largo del proyecto aprendimos a organizar el código con el patrón MVC, lo que hizo que cada parte del sistema fuera independiente y más fácil de mantener y depurar.

Ángel Adrián se encargó de definir la arquitectura general del proyecto: los paquetes, las clases de modelo y cómo se conectan entre sí siguiendo el patrón MVC. Felipe González desarrolló la lógica de negocio a través de los controladores, implementando las validaciones y los flujos de venta, inventario, compras y devoluciones. Kevin David construyó las utilidades transversales del sistema: la conexión a MySQL, el lector de JSON, el manejador de errores, los reportes y el cierre de caja.

Uno de los aprendizajes más importantes fue entender cómo Java se comunica con MySQL a través de JDBC. Antes de este proyecto, conocíamos los conceptos de bases de datos de forma teórica, pero al implementar las clases DAO y ver cómo una acción del usuario en la pantalla termina ejecutando un INSERT o un UPDATE en la base de datos, la conexión entre ambas materias se volvió completamente clara.

Como equipo, concluimos que la combinación de una base de datos bien diseñada y una aplicación estructurada con buenos patrones de diseño permite construir software funcional, fácil de mantener y preparado para crecer. MegaPOS cumple con las funciones principales de un punto de venta para una tienda de abarrotes: venta de productos, control de inventario, entradas de mercancía, devoluciones, generación y reimpresión de tickets, reportes y cierre de caja. Con mejoras futuras, el sistema podría adaptarse a una tienda real.